

```
SQL Plus
Enter user-name: SYSTEM
Enter password:
Last Successful login time: Fri Dec 05 2025 18:13:46 +02:00

Connected to:
Oracle Database 23ai Free Release 23.0.0.0.0 - Develop, Learn, and Run for Free
Version 23.9.0.25.07

SQL> SELECT name, open_mode FROM v$pdb;

NAME
-----
OPEN_MODE
-----
PDB$SEED
READ ONLY

FREEPDB1
READ WRITE

SQL> ALTER SESSION SET CONTAINER = FREEPDB1;

Session altered.

SQL> CREATE USER grpA_divine_cropdiseases_db IDENTIFIED BY Divine;

User created.

SQL> GRANT DBA TO grpA_divine_cropdiseases_db;

Grant succeeded.

SQL>
```

Creation of the user

```
SQL Plus

SQL> CREATE SEQUENCE seq_users START WITH 1 NOCACHE;

Sequence created.

SQL> CREATE SEQUENCE seq_crops START WITH 1 NOCACHE;

Sequence created.

SQL> CREATE SEQUENCE seq_diseases START WITH 1 NOCACHE;

Sequence created.

SQL> CREATE SEQUENCE seq_reports START WITH 1 NOCACHE;

Sequence created.

SQL> CREATE SEQUENCE seq_analysis START WITH 1 NOCACHE;

Sequence created.

SQL> CREATE SEQUENCE seq_recs START WITH 1 NOCACHE;

Sequence created.

SQL> CREATE SEQUENCE seq_alerts START WITH 1 NOCACHE;

Sequence created.

SQL> CREATE SEQUENCE seq_audit START WITH 1 NOCACHE;

Sequence created.

SQL>
SQL> -- ===== TABLES =====
SQL>
SQL> CREATE TABLE users (
2  user_id      NUMBER(10) PRIMARY KEY,
3  full_name    VARCHAR2(100) NOT NULL,
4  phone        VARCHAR2(20),
5  role         VARCHAR2(20) DEFAULT 'Farmer' CHECK (role IN ('Farmer','Officer','Admin')),
6  location     VARCHAR2(100),
7  created_at   DATE DEFAULT SYSDATE
8 );
```

Creating sequences

```
SQL Plus
212 COMMIT;
213
214
215 EXCEPTION
216 WHEN OTHERS THEN
217     v_errmsg := SQLERRM;
218
219     INSERT INTO audit_log(
220         log_id, user_id, action, object_type, object_id, status, notes
221     )
222     VALUES (
223         seq_audit.NEXTVAL,
224         p_user_id,
225         'INSERT_REPORT',
226         'REPORT',
227         v_rid,
228         'FAILED',
229         v_errmsg
230     );
231
232 ROLLBACK;
233 RAISE;
234 END insert_report;
235
236 END crop_pkg;
237 /
Package body created.
SQL>
```

```
SQL Plus
ORA-00931: missing identifier
Help: https://docs.oracle.com/error-help/db/ora-00931/

SQL> CREATE TABLE reports (
2     report_id      NUMBER(12) PRIMARY KEY,
3     user_id        NUMBER(10) NOT NULL,
4     crop_id        NUMBER(10) NOT NULL,
5     symptoms_observed VARCHAR2(1000) NOT NULL,
6     severity        NUMBER(3) DEFAULT 5 CHECK (severity BETWEEN 1 AND 10),
7     report_location VARCHAR2(200),
8     report_date     DATE DEFAULT SYSDATE,
9     status          VARCHAR2(20) DEFAULT 'PENDING' CHECK (status IN ('PENDING', 'ANALYZED', 'RESPONDED')),
10    CONSTRAINT fk_reports_user FOREIGN KEY (user_id) REFERENCES users(user_id),
11    CONSTRAINT fk_reports_crop FOREIGN KEY (crop_id) REFERENCES crops(crop_id)
12 );
Table created.

SQL> CREATE TABLE analysis (
2     analysis_id    NUMBER(12) PRIMARY KEY,
3     report_id      NUMBER(12) NOT NULL UNIQUE,
4     disease_id     NUMBER(10),
5     risk_level     VARCHAR2(20) CHECK (risk_level IN ('LOW', 'MEDIUM', 'HIGH', 'CRITICAL')),
6     confidence_score NUMBER(5,2) CHECK (confidence_score BETWEEN 0 AND 100),
7     analysis_date   DATE DEFAULT SYSDATE,
8     CONSTRAINT fk_analysis_report FOREIGN KEY (report_id) REFERENCES reports(report_id) ON DELETE CASCADE,
9     CONSTRAINT fk_analysis_disease FOREIGN KEY (disease_id) REFERENCES diseases(disease_id)
10 );
Table created.

SQL> CREATE TABLE recommendations (
2     rec_id         NUMBER(12) PRIMARY KEY,
3     analysis_id    NUMBER(12) NOT NULL,
4     recommendation_text VARCHAR2(4000),
5     created_at     DATE DEFAULT SYSDATE,
6     CONSTRAINT fk_rec_analysis FOREIGN KEY (analysis_id) REFERENCES analysis(analysis_id) ON DELETE CASCADE
7 );
Table created.

SQL>
```

Package body creation and table creation

```

SQL Plus
185 v_errmsg VARCHAR2(4000);
186 BEGIN
187     IF p_symptoms IS NULL OR TRIM(p_symptoms) = '' THEN
188         RAISE_APPLICATION_ERROR(-20002, 'Symptoms required.');
```

```

189     END IF;
190
191     v_rid := seq_reports.NEXTVAL;
192
193     INSERT INTO reports(
194         report_id, user_id, crop_id, symptoms_observed, severity, report_location, report_date, status
195     )
196     VALUES (
197         v_rid, p_user_id, p_crop_id, p_symptoms, NVL(p_severity,5), p_location, SYSDATE, 'PENDING'
198     );
199
200     INSERT INTO audit_log(
201         log_id, user_id, action, object_type, object_id, status, notes
202     )
203     VALUES (
204         seq_audit.NEXTVAL,
205         p_user_id,
206         'INSERT_REPORT',
207         'REPORT',
208         v_rid,
209         'SUCCESS',
210         'Report inserted via insert_report'
211     );
212
213     COMMIT;
214
215 EXCEPTION
216     WHEN OTHERS THEN
217         v_errmsg := SQLERRM;
218
219     INSERT INTO audit_log(
220         log_id, user_id, action, object_type, object_id, status, notes
221     )
222     VALUES (
223         seq_audit.NEXTVAL,
224         p_user_id,
225         'INSERT_REPORT',
226         'REPORT',
227         v_rid,
```

Insertion of data in tables

```

SQL Plus
8 );
9 END;
10 /

PL/SQL procedure successfully completed.

SQL>
SQL> -- Check what report was created
SQL> SELECT report_id, user_id, crop_id, status FROM reports WHERE user_id = 1 ORDER BY report_id DESC;
```

REPORT_ID	USER_ID	CROP_ID	STATUS
201	1	1	PENDING
199	1	2	PENDING
198	1	1	PENDING
197	1	2	PENDING
196	1	1	PENDING
195	1	2	PENDING
194	1	1	PENDING
193	1	2	PENDING
192	1	1	PENDING
191	1	2	PENDING
189	1	2	PENDING

REPORT_ID	USER_ID	CROP_ID	STATUS
188	1	1	PENDING
187	1	2	PENDING
186	1	1	PENDING
185	1	2	PENDING
184	1	1	PENDING
183	1	2	PENDING
182	1	1	PENDING
181	1	2	PENDING
179	1	2	PENDING
178	1	1	PENDING
177	1	2	PENDING

REPORT_ID	USER_ID	CROP_ID	STATUS
176	1	1	PENDING
175	1	2	PENDING
174	1	1	PENDING

```

9
10 EXCEPTION
11 WHEN NO_DATA_FOUND THEN
12 RETURN 'Unknown Disease';
13 WHEN OTHERS THEN
14 RETURN 'Error';
15 END get_disease_name;
16 /

Function created.

SQL> CREATE OR REPLACE FUNCTION get_avg_severity_by_location(
2 p_location IN VARCHAR2
3 ) RETURN NUMBER IS
4 v_avg NUMBER;
5 BEGIN
6 SELECT NVL(AVG(severity), 0)
7 INTO v_avg
8 FROM reports
9 WHERE REPORT_LOCATION = p_location;
10
11 RETURN ROUND(v_avg, 2);
12
13 EXCEPTION
14 WHEN OTHERS THEN
15 RETURN 0;
16 END get_avg_severity_by_location;
17 /

Function created.

SQL> CREATE OR REPLACE PACKAGE report_analytics_pkg IS
2 -- Public procedures and functions
3 PROCEDURE generate_location_report(p_location IN VARCHAR2);
4 PROCEDURE bulk_update_severity(p_increment IN NUMBER);
5 FUNCTION get_top_diseases(p_limit IN NUMBER DEFAULT 5) RETURN SYS_REFCURSOR;
6 PROCEDURE process_reports_with_cursor;
7 END report_analytics_pkg;
8 /

Package created.

SQL>

```

Package and package body created

```

90
91 RETURN v_cursor;
92 END get_top_diseases;
93
94 -- Process reports with bulk collect
95 PROCEDURE process_reports_with_cursor IS
96 TYPE t_report_ids IS TABLE OF NUMBER;
97 TYPE t_severities IS TABLE OF NUMBER;
98
99 v_report_ids t_report_ids;
100 v_severities t_severities;
101
102 CURSOR c_pending IS
103 SELECT report_id, severity
104 FROM reports
105 WHERE status = 'PENDING';
106 BEGIN
107 OPEN c_pending;
108 FETCH c_pending BULK COLLECT INTO v_report_ids, v_severities LIMIT 100;
109 CLOSE c_pending;
110
111 DBMS_OUTPUT.PUT_LINE('Fetched ' || v_report_ids.COUNT || ' pending reports');
112
113 -- Process in bulk
114 FOR i IN 1..LEAST(v_report_ids.COUNT, 5) LOOP
115 DBMS_OUTPUT.PUT_LINE('Processing report ' || v_report_ids(i) ||
116 ' with severity ' || v_severities(i));
117 END LOOP;
118
119 EXCEPTION
120 WHEN OTHERS THEN
121 IF c_pending%ISOPEN THEN
122 CLOSE c_pending;
123 END IF;
124 DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
125 END process_reports_with_cursor;
126
127 END report_analytics_pkg;
128 /

Package body created.

SQL>

```

```
Select SQL Plus
SQL> SELECT a.analysis_id, a.report_id, a.disease_id, a.risk_level, a.confidence_score
  2 FROM analysis a
  3 WHERE a.report_id = 201;

ANALYSIS_ID  REPORT_ID  DISEASE_ID  RISK_LEVEL  CONFIDENCE_SCORE
-----
          1          201           2    HIGH              75

SQL>
SQL> -- Check recommendations
SQL> SELECT r.rec_id, r.analysis_id, r.recommendation_text
  2 FROM recommendations r
  3 JOIN analysis a ON r.analysis_id = a.analysis_id
  4 WHERE a.report_id = 201;

REC_ID  ANALYSIS_ID
-----
          1          1
RECOMMENDATION_TEXT
-----
For disease: Early Blight. Isolate affected plants, apply recommended treatment,
maintain sanitation. Severity=6

SQL>
SQL> -- Check if report status was updated
SQL> SELECT report_id, status FROM reports WHERE report_id = 201;

REPORT_ID  STATUS
-----
          201 ANALYZED

SQL>
SQL> -- Check audit log
SQL> SELECT log_id, action, object_type, object_id, status, notes
  2 FROM audit_log
  3 WHERE object_id = '201'
  4 ORDER BY action_time DESC;_
```

```
SQL Plus
GENERATE_ANALYSIS
REPORT

LOG_ID
-----
ACTION
-----
OBJECT_TYPE
-----
OBJECT_ID  STATUS
-----
NOTES
-----
201          SUCCESS
Analysis generated

LOG_ID
-----
ACTION
-----
OBJECT_TYPE
-----
OBJECT_ID  STATUS
-----
NOTES
-----
1
INSERT_REPORT
REPORT

LOG_ID
-----
ACTION
-----
OBJECT_TYPE
-----
OBJECT_ID  STATUS
-----
NOTES
-----
201          SUCCESS
Report inserted via insert_report
```

```
SQL Plus
SQL> SET SERVEROUTPUT ON
SQL> SELECT 'EXISTING TABLES' as info FROM dual;

INFO
-----
EXISTING TABLES

SQL> SELECT table_name FROM user_tables ORDER BY table_name;

TABLE_NAME
-----
ANALYSIS
AUDIT_LOG
CROPS
DISEASES
OUTBREAK_ALERTS
RECOMMENDATIONS
REPORTS
USERS

8 rows selected.

SQL>
SQL> SELECT 'EXISTING SEQUENCES' as info FROM dual;

INFO
-----
EXISTING SEQUENCES

SQL> SELECT sequence_name FROM user_sequences ORDER BY sequence_name;
```

showing created tables as well as sequences

```
SQL Plus

INFO
-----
EXISTING TABLES

SQL> SELECT table_name FROM user_tables ORDER BY table_name;

TABLE_NAME
-----
ANALYSIS
AUDIT_LOG
CROPS
DISEASES
OUTBREAK_ALERTS
RECOMMENDATIONS
REPORTS
USERS

8 rows selected.

SQL>
SQL> SELECT 'EXISTING SEQUENCES' as info FROM dual;

INFO
-----
EXISTING SEQUENCES

SQL> SELECT sequence_name FROM user_sequences ORDER BY sequence_name;

SEQUENCE_NAME
-----
SEQ_ALERTS
SEQ_ANALYSIS
SEQ_AUDIT
SEQ_CROPS
SEQ_DISEASES
SEQ_RECS
SEQ_REPORTS
SEQ_USERS

8 rows selected.

SQL>
```

```
SQL Plus
SQL> INSERT INTO users(user_id, full_name, phone, role, location, created_at)
  2 VALUES (9, 'Agnes Mukamazimpaka', '0788901234', 'Farmer', 'Rubavu', SYSDATE - 70);

1 row created.

SQL>
SQL> INSERT INTO users(user_id, full_name, phone, role, location, created_at)
  2 VALUES (10, 'David Kalisa', '0789012345', 'Officer', 'Muhanga', SYSDATE - 65);

1 row created.

SQL>
SQL> INSERT INTO users(user_id, full_name, phone, role, location, created_at)
  2 VALUES (11, 'Claire Ingabire', '0789123456', 'Farmer', 'Ngoma', SYSDATE - 60);

1 row created.

SQL>
SQL> INSERT INTO users(user_id, full_name, phone, role, location, created_at)
  2 VALUES (12, 'Patrick Bizimungu', '0789234567', 'Farmer', 'Bugesera', SYSDATE - 55);

1 row created.

SQL>
SQL> INSERT INTO users(user_id, full_name, phone, role, location, created_at)
  2 VALUES (13, 'Esperance Nyiramana', '0789345678', 'Farmer', 'Kamonyi', SYSDATE - 50);

1 row created.

SQL>
SQL> INSERT INTO users(user_id, full_name, phone, role, location, created_at)
  2 VALUES (14, 'Eric Nsabimana', '0789456789', 'Admin', 'Kigali', SYSDATE - 45);

1 row created.

SQL>
SQL> INSERT INTO users(user_id, full_name, phone, role, location, created_at)
  2 VALUES (15, 'Francine Umutoni', '0789567890', 'Farmer', 'Rulindo', SYSDATE - 40);

1 row created.

SQL>
SQL> COMMIT;
```

```
SQL Plus
SQL>
SQL> INSERT INTO users(user_id, full_name, phone, role, location, created_at)
  2 VALUES (10, 'David Kalisa', '0789012345', 'Officer', 'Muhanga', SYSDATE - 65);

1 row created.

SQL>
SQL> INSERT INTO users(user_id, full_name, phone, role, location, created_at)
  2 VALUES (11, 'Claire Ingabire', '0789123456', 'Farmer', 'Ngoma', SYSDATE - 60);

1 row created.

SQL>
SQL> INSERT INTO users(user_id, full_name, phone, role, location, created_at)
  2 VALUES (12, 'Patrick Bizimungu', '0789234567', 'Farmer', 'Bugesera', SYSDATE - 55);

1 row created.

SQL>
SQL> INSERT INTO users(user_id, full_name, phone, role, location, created_at)
  2 VALUES (13, 'Esperance Nyiramana', '0789345678', 'Farmer', 'Kamonyi', SYSDATE - 50);

1 row created.

SQL>
SQL> INSERT INTO users(user_id, full_name, phone, role, location, created_at)
  2 VALUES (14, 'Eric Nsabimana', '0789456789', 'Admin', 'Kigali', SYSDATE - 45);

1 row created.

SQL>
SQL> INSERT INTO users(user_id, full_name, phone, role, location, created_at)
  2 VALUES (15, 'Francine Umutoni', '0789567890', 'Farmer', 'Rulindo', SYSDATE - 40);

1 row created.

SQL>
SQL> COMMIT;

Commit complete.

SQL>
```

```

SQL Plus
SQL> INSERT INTO diseases(disease_id, crop_id, disease_name, typical_symptoms, contagion_level)
  2 VALUES (23, 8, 'Blister Blight', 'blisters; concentric rings; leaf drop', 'HIGH');

1 row created.

SQL>
SQL> INSERT INTO diseases(disease_id, crop_id, disease_name, typical_symptoms, contagion_level)
  2 VALUES (24, 8, 'Gray Blight', 'gray spots; stem canker; dieback', 'MEDIUM');

1 row created.

SQL>
SQL> -- Sweet Potato diseases
SQL> INSERT INTO diseases(disease_id, crop_id, disease_name, typical_symptoms, contagion_level)
  2 VALUES (25, 8, 'Sweet Potato Virus Disease', 'stunting; chlorosis; vein clearing; leaf distortion', 'HIGH');

1 row created.

SQL>
SQL> INSERT INTO diseases(disease_id, crop_id, disease_name, typical_symptoms, contagion_level)
  2 VALUES (26, 8, 'Sweet Potato Weevil', 'tunneling; root damage; wilting', 'MEDIUM');

1 row created.

SQL>
SQL> -- Sorghum diseases
SQL> INSERT INTO diseases(disease_id, crop_id, disease_name, typical_symptoms, contagion_level)
  2 VALUES (27, 10, 'Sorghum Anthracnose', 'anthracnose lesions; leaf spots; stalk rot', 'MEDIUM');

1 row created.

SQL>
SQL> INSERT INTO diseases(disease_id, crop_id, disease_name, typical_symptoms, contagion_level)
  2 VALUES (28, 10, 'Grain Mold', 'grain discoloration; mold growth; reduced quality', 'LOW');

1 row created.

SQL>
SQL> COMMIT;

Commit complete.

SQL>

```

```

SQL Plus
37 -- Random symptom
38 v_symptom := v_symptoms(MOD(i, v_symptoms.COUNT) + 1);
39
40 -- Random location
41 v_location := v_locations(MOD(i, v_locations.COUNT) + 1);
42
43 -- Random user (1-15)
44 v_user_id := MOD(i, 15) + 1;
45
46 -- Random crop (1-10)
47 v_crop_id := MOD(i, 10) + 1;
48
49 -- Random severity (1-10)
50 v_severity := TRUNC(DBMS_RANDOM.VALUE(1, 11));
51
52 -- Random date within last 180 days
53 v_days_ago := TRUNC(DBMS_RANDOM.VALUE(0, 180));
54
55 INSERT INTO reports(report_id, user_id, crop_id, symptoms_observed, severity, REPORT_LOCATION, report_date, status)
56 VALUES (
57     seq_reports.NEXTVAL,
58     v_user_id,
59     v_crop_id,
60     v_symptom,
61     v_severity,
62     v_location,
63     SYSDATE - v_days_ago,
64     CASE
65         WHEN MOD(i, 5) = 0 THEN 'ANALYZED'
66         WHEN MOD(i, 10) = 0 THEN 'RESPONDED'
67         ELSE 'PENDING'
68     END
69 );
70 END LOOP;
71
72 COMMIT;
73 DBMS_OUTPUT.PUT_LINE('Successfully inserted 300 additional reports');
74 END;
75 /
Successfully inserted 300 additional reports

PL/SQL procedure successfully completed.

```

```
SQL Plus
18
19 -- Calculate confidence (simplified)
20 v_confidence := TRUNC(DBMS_RANDOM.VALUE(40, 95));
21
22 -- Determine risk level
23 IF rec.severity >= 8 OR v_confidence >= 70 THEN
24   v_risk := 'HIGH';
25 ELSIF rec.severity >= 5 OR v_confidence >= 50 THEN
26   v_risk := 'MEDIUM';
27 ELSIF v_confidence >= 30 THEN
28   v_risk := 'LOW';
29 ELSE
30   v_risk := 'LOW';
31 END IF;
32
33 -- Insert analysis
34 INSERT INTO analysis(analysis_id, report_id, disease_id, risk_level, confidence_score, analysis_date)
35 VALUES (seq_analysis.NEXTVAL, rec.report_id, v_disease_id, v_risk, v_confidence, SYSDATE);
36
37 -- Insert recommendation
38 INSERT INTO recommendations(rec_id, analysis_id, recommendation_text, created_at)
39 VALUES (
40   seq_recs.NEXTVAL,
41   seq_analysis.CURRVAL,
42   'Recommended treatment for severity level ' || rec.severity || '. Monitor closely and apply appropriate fungicide/pesticide.',
43   SYSDATE
44 );
45
46 EXCEPTION
47 WHEN NO_DATA_FOUND THEN
48   NULL; -- Skip if no disease found for this crop
49 END;
50 END LOOP;
51
52 COMMIT;
53 DBMS_OUTPUT.PUT_LINE('Analysis generated for analyzed reports');
54 END;
55 /
Analysis generated for analyzed reports
PL/SQL procedure successfully completed.
SQL>
```

```
SQL Plus
52 COMMIT;
53 DBMS_OUTPUT.PUT_LINE('Analysis generated for analyzed reports');
54 END;
55 /
Analysis generated for analyzed reports
PL/SQL procedure successfully completed.

SQL> DECLARE
2  CURSOR c_outbreaks IS
3  SELECT d.disease_id, r.REPORT_LOCATION, COUNT(*) as case_count
4  FROM reports r
5  JOIN analysis a ON r.report_id = a.report_id
6  JOIN diseases d ON a.disease_id = d.disease_id
7  WHERE r.report_date >= SYSDATE - 30
8  GROUP BY d.disease_id, r.REPORT_LOCATION
9  HAVING COUNT(*) >= 5;
10
11 v_alert_level VARCHAR2(20);
12 BEGIN
13 FOR rec IN c_outbreaks LOOP
14   -- Determine alert level based on case count
15   IF rec.case_count >= 15 THEN
16     v_alert_level := 'OUTBREAK';
17   ELSIF rec.case_count >= 10 THEN
18     v_alert_level := 'ADVISORY';
19   ELSE
20     v_alert_level := 'WATCH';
21   END IF;
22
23   INSERT INTO outbreak_alerts(alert_id, disease_id, location, number_of_cases, alert_date, alert_level)
24   VALUES (seq_alerts.NEXTVAL, rec.disease_id, rec.REPORT_LOCATION, rec.case_count, SYSDATE, v_alert_level);
25 END LOOP;
26
27 COMMIT;
28 DBMS_OUTPUT.PUT_LINE('Outbreak alerts generated');
29 END;
30 /
Outbreak alerts generated
PL/SQL procedure successfully completed.

SQL> DECLARE
```

```
SQL Plus
PL/SQL procedure successfully completed.

SQL> DECLARE
2   v_actions SYS.ODCIVARCHAR2LIST := SYS.ODCIVARCHAR2LIST(
3     'LOGIN', 'LOGOUT', 'VIEW_REPORT', 'UPDATE_REPORT',
4     'GENERATE_ANALYSIS', 'VIEW_DASHBOARD'
5   );
6   v_action VARCHAR2(50);
7   v_user_id NUMBER;
8 BEGIN
9   FOR i IN 1..100 LOOP
10    v_action := v_actions(MOD(i, v_actions.COUNT) + 1);
11    v_user_id := MOD(i, 15) + 1;
12
13    INSERT INTO audit_log(log_id, user_id, action, object_type, object_id, status, notes, action_time)
14    VALUES (
15      seq_audit.NEXTVAL,
16      v_user_id,
17      v_action,
18      'SYSTEM',
19      TO_CHAR(i),
20      CASE WHEN MOD(i, 20) = 0 THEN 'FAILED' ELSE 'SUCCESS' END,
21      'Automated audit entry for testing',
22      SYSDATE - TRUNC(DBMS_RANDOM.VALUE(0, 90))
23    );
24  END LOOP;
25
26  COMMIT;
27  DBMS_OUTPUT.PUT_LINE('Audit log entries generated');
28 END;
29 /
Audit log entries generated

PL/SQL procedure successfully completed.

SQL> -
```

```
SQL Plus
21   'Automated audit entry for testing',
22   SYSDATE - TRUNC(DBMS_RANDOM.VALUE(0, 90))
23 );
24 END LOOP;
25
26 COMMIT;
27 DBMS_OUTPUT.PUT_LINE('Audit log entries generated');
28 END;
29 /
Audit log entries generated

PL/SQL procedure successfully completed.

SQL> SELECT 'USERS' as table_name, COUNT(*) as row_count FROM users
2 UNION ALL
3 SELECT 'CROPS', COUNT(*) FROM crops
4 UNION ALL
5 SELECT 'DISEASES', COUNT(*) FROM diseases
6 UNION ALL
7 SELECT 'REPORTS', COUNT(*) FROM reports
8 UNION ALL
9 SELECT 'ANALYSIS', COUNT(*) FROM analysis
10 UNION ALL
11 SELECT 'RECOMMENDATIONS', COUNT(*) FROM recommendations
12 UNION ALL
13 SELECT 'OUTBREAK_ALERTS', COUNT(*) FROM outbreak_alerts
14 UNION ALL
15 SELECT 'AUDIT_LOG', COUNT(*) FROM audit_log;

TABLE_NAME      ROW_COUNT
-----
USERS            15
CROPS            10
DISEASES         28
REPORTS         501
ANALYSIS         61
RECOMMENDATIONS  61
OUTBREAK_ALERTS  0
AUDIT_LOG       102

8 rows selected.

SQL>
```

```
SQL Plus
6 FROM reports
7 GROUP BY TO_CHAR(report_date, 'YYYY-MM')
8 ORDER BY month DESC;

MONTH      REPORT_COUNT AVG_SEVERITY ANALYZED_COUNT
-----
2025-12          41    5.63414634           3
2025-11         218    5.38973394           8
2025-10          52    5.82692308          11
2025-09          55    5.78181818          11
2025-08          56    5.46428571          12
2025-07          45    5.35555556           8
2025-06          34    5.41176471           8

7 rows selected.

SQL>
SQL> -- 10.12 Foreign Key Verification
SQL> SELECT
2   'Reports without valid users' as check_name,
3   COUNT(*) as violation_count
4 FROM reports r
5 WHERE NOT EXISTS (SELECT 1 FROM users u WHERE u.user_id = r.user_id)
6 UNION ALL
7 SELECT
8   'Analysis without valid reports',
9   COUNT(*)
10 FROM analysis a
11 WHERE NOT EXISTS (SELECT 1 FROM reports r WHERE r.report_id = a.report_id)
12 UNION ALL
13 SELECT
14   'Diseases without valid crops',
15   COUNT(*)
16 FROM diseases d
17 WHERE NOT EXISTS (SELECT 1 FROM crops c WHERE c.crop_id = d.crop_id);

CHECK_NAME      VIOLATION_COUNT
-----
Reports without valid users      0
Analysis without valid reports    0
Diseases without valid crops      0

SQL>
```

```
SQL Plus
10 FROM analysis a
11 WHERE NOT EXISTS (SELECT 1 FROM reports r WHERE r.report_id = a.report_id)
12 UNION ALL
13 SELECT
14   'Diseases without valid crops',
15   COUNT(*)
16 FROM diseases d
17 WHERE NOT EXISTS (SELECT 1 FROM crops c WHERE c.crop_id = d.crop_id);

CHECK_NAME      VIOLATION_COUNT
-----
Reports without valid users      0
Analysis without valid reports    0
Diseases without valid crops      0

SQL> SELECT
2   'Users with NULL phone' as check_item,
3   COUNT(*) as null_count
4 FROM users WHERE phone IS NULL
5 UNION ALL
6 SELECT
7   'Reports with NULL severity',
8   COUNT(*)
9 FROM reports WHERE severity IS NULL
10 UNION ALL
11 SELECT
12   'Analysis with NULL disease_id',
13   COUNT(*)
14 FROM analysis WHERE disease_id IS NULL
15 UNION ALL
16 SELECT
17   'Reports with NULL location',
18   COUNT(*)
19 FROM reports WHERE REPORT_LOCATION IS NULL;

CHECK_ITEM      NULL_COUNT
-----
Users with NULL phone      0
Reports with NULL severity    0
Analysis with NULL disease_id 0
Reports with NULL location    0

SQL>
```

```

SQL Plus
17 'Reports with NULL location',
18 COUNT(*)
19 FROM reports WHERE REPORT_LOCATION IS NULL;

CHECK_ITEM                                NULL_COUNT
-----
Users with NULL phone                     0
Reports with NULL severity                 0
Analysis with NULL disease_id              0
Reports with NULL location                 0

SQL> SELECT
2 'Severity out of range (1-10)' as constraint_check,
3 COUNT(*) as violation_count
4 FROM reports
5 WHERE severity NOT BETWEEN 1 AND 10
6 UNION ALL
7 SELECT
8 'Invalid report status',
9 COUNT(*)
10 FROM reports
11 WHERE status NOT IN ('PENDING', 'ANALYZED', 'RESPONDED')
12 UNION ALL
13 SELECT
14 'Invalid risk level',
15 COUNT(*)
16 FROM analysis
17 WHERE risk_level NOT IN ('LOW', 'MEDIUM', 'HIGH', 'CRITICAL')
18 UNION ALL
19 SELECT
20 'Confidence score out of range (0-100)',
21 COUNT(*)
22 FROM analysis
23 WHERE confidence_score NOT BETWEEN 0 AND 100;

CONSTRAINT_CHECK                          VIOLATION_COUNT
-----
Severity out of range (1-10)              0
Invalid report status                     0
Invalid risk level                        0
Confidence score out of range (0-100)     0

SQL>

```

```

SQL Plus
2
3 /
BEGIN EXECUTE IMMEDIATE 'CREATE TABLE error_log (
error_id NUMBER PRIMARY KEY, error_code VARCHAR2(50), error_message VARCHAR2(4000), error_p
rocedure VARCHAR2(200), error_date DATE DEFAULT SYSDATE, additional_info VARCHAR2(2000) ); DBMS_OUTPUT.PUT_LINE('Error log table created');EXCEPTION WHEN
OTHERS THEN IF SQLCODE = -955 THEN DBMS_OUTPUT.PUT_LINE('Error log table already exists'); ELSE RAISE; END IF;END;/CREATE SEQUENCE
seq_error_log START WITH 1 NOCACHE;

*
ERROR at line 1:
ORA-06550: line 1, column 508:
PLS-00103: Encountered the symbol "/"
Help: https://docs.oracle.com/error-help/db/ora-06550/

SQL> BEGIN
2 EXECUTE IMMEDIATE 'CREATE TABLE error_log (
3 error_id NUMBER PRIMARY KEY,
4 error_code VARCHAR2(50),
5 error_message VARCHAR2(4000),
6 error_procedure VARCHAR2(200),
7 error_date DATE DEFAULT SYSDATE,
8 additional_info VARCHAR2(2000)
9 );'
10 DBMS_OUTPUT.PUT_LINE('Error log table created');
11 EXCEPTION
12 WHEN OTHERS THEN
13 IF SQLCODE = -955 THEN
14 DBMS_OUTPUT.PUT_LINE('Error log table already exists');
15 ELSE
16 RAISE;
17 END IF;
18 END;
19 /
Error log table created

PL/SQL procedure successfully completed.

SQL> CREATE SEQUENCE seq_error_log START WITH 1 NOCACHE;

Sequence created.

SQL>

```

```
SQL Plus
15 v_cutoff_date := SYSDATE - p_days_old;
16
17 -- Delete old reports
18 DELETE FROM reports
19 WHERE report_date < v_cutoff_date
20 AND status = 'RESPONDED';
21
22 p_delete_count := SQL%ROWCOUNT;
23
24 -- Log the operation
25 INSERT INTO audit_log
26 VALUES (
27     seq_audit.NEXTVAL,
28     NULL,
29     'BULK_DELETE',
30     'REPORT',
31     'MULTIPLE',
32     'SUCCESS',
33     'Deleted ' || p_delete_count || ' reports older than ' || p_days_old || ' days',
34     SYSDATE
35 );
36
37 COMMIT;
38 p_success := TRUE;
39 p_message := p_delete_count || ' old reports deleted successfully';
40
41 EXCEPTION
42 WHEN e_invalid_days THEN
43     ROLLBACK;
44     p_success := FALSE;
45     p_message := 'Days must be greater than 0';
46     p_delete_count := 0;
47 WHEN OTHERS THEN
48     ROLLBACK;
49     p_success := FALSE;
50     p_message := 'Error: ' || SQLERRM;
51     p_delete_count := 0;
52 END delete_old_reports;
53 /
Procedure created.
SQL>
```

```
SQL Plus
15 END;
16
17 -- Calculate risk score (0-100 scale)
18 v_risk_score := ((p_severity * 0.6) + (p_confidence * 0.4)) * v_contagion_weight;
19
20 -- Cap at 100
21 IF v_risk_score > 100 THEN
22     v_risk_score := 100;
23 END IF;
24
25 RETURN ROUND(v_risk_score, 2);
26
27 EXCEPTION
28 WHEN OTHERS THEN
29     RETURN 0;
30 END calculate_risk_score;
31 /
Function created.
SQL> CREATE OR REPLACE FUNCTION validate_phone(p_phone IN VARCHAR2) RETURN BOOLEAN IS
2 BEGIN
3     -- Rwanda phone format: 07XX XXX XXX or +25078X XXX XXX
4     IF p_phone IS NULL THEN
5         RETURN FALSE;
6     END IF;
7
8     -- Remove spaces
9     IF REGEXP_LIKE(REPLACE(p_phone, ' ', ''), '^(07[2-9][0-9]{7}|\\+25078[0-9]{7})$') THEN
10         RETURN TRUE;
11     ELSE
12         RETURN FALSE;
13     END IF;
14
15 EXCEPTION
16 WHEN OTHERS THEN
17     RETURN FALSE;
18 END validate_phone;
19 /
Function created.
SQL>
```

```

SQL Plus
5  v_count NUMBER;
6  BEGIN
7      IF p_status IS NULL THEN
8          SELECT COUNT(*) INTO v_count
9              FROM reports
10             WHERE user_id = p_user_id;
11      ELSE
12          SELECT COUNT(*) INTO v_count
13              FROM reports
14              WHERE user_id = p_user_id
15                AND status = p_status;
16      END IF;
17
18      RETURN v_count;
19
20 EXCEPTION
21     WHEN OTHERS THEN
22         RETURN 0;
23 END get_user_report_count;
24 /

Function created.

SQL> CREATE OR REPLACE FUNCTION get_disease_name(p_disease_id IN NUMBER) RETURN VARCHAR2 IS
2  v_name VARCHAR2(100);
3  BEGIN
4      SELECT disease_name INTO v_name
5          FROM diseases
6          WHERE disease_id = p_disease_id;
7
8      RETURN v_name;
9
10 EXCEPTION
11     WHEN NO_DATA_FOUND THEN
12         RETURN 'Unknown Disease';
13     WHEN OTHERS THEN
14         RETURN 'Error';
15 END get_disease_name;
16 /

Function created.

SQL>

```

```

SQL Plus
39  is_allowed,
40  denial_reason,
41  old_values,
42  new_values,
43  additional_info
44  )
45  VALUES (
46      v_id,
47      'REPORTS',
48      p_action,
49      p_user,
50      SYSDATE,
51      TRIM(v_day),
52      v_is_holiday,
53      p_status,
54      p_reason,
55      NULL,
56      NULL,
57      NULL
58  );
59
60  COMMIT;
61  RETURN v_id;
62
63 EXCEPTION
64     WHEN OTHERS THEN
65         v_error := SQLERRM;
66         ROLLBACK;
67         -- Log error to error_log if exists
68         BEGIN
69             INSERT INTO error_log(error_id, error_code, error_message, error_procedure, error_date)
70                 VALUES (seq_error_log.NEXTVAL, 'LOG_ACTION_ERROR', v_error, 'log_action', SYSDATE);
71             COMMIT;
72         EXCEPTION
73             WHEN OTHERS THEN NULL;
74         END;
75     RETURN -1;
76 END;
77 /

Function created.

SQL>

```

```
SQL Plus
Package body created.

SQL> CREATE OR REPLACE VIEW v_report_rankings AS
2  SELECT
3      r.report_id,
4      r.REPORT_LOCATION,
5      c.crop_name,
6      r.severity,
7      r.report_date,
8      -- ROW_NUMBER: Unique sequential number
9      ROW_NUMBER() OVER (ORDER BY r.severity DESC, r.report_date DESC) as severity_row_num,
10     -- RANK: Same rank for ties, gaps in sequence
11     RANK() OVER (ORDER BY r.severity DESC) as severity_rank,
12     -- DENSE_RANK: Same rank for ties, no gaps
13     DENSE_RANK() OVER (ORDER BY r.severity DESC) as severity_dense_rank,
14     -- PARTITION BY location
15     ROW_NUMBER() OVER (PARTITION BY r.REPORT_LOCATION ORDER BY r.severity DESC) as location_rank,
16     -- LAG: Previous report severity
17     LAG(r.severity, 1, 0) OVER (PARTITION BY r.REPORT_LOCATION ORDER BY r.report_date) as prev_severity,
18     -- LEAD: Next report severity
19     LEAD(r.severity, 1, 0) OVER (PARTITION BY r.REPORT_LOCATION ORDER BY r.report_date) as next_severity,
20     -- Moving average
21     AVG(r.severity) OVER (
22         PARTITION BY r.REPORT_LOCATION
23         ORDER BY r.report_date
24         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
25     ) as moving_avg_severity,
26     -- Cumulative count
27     COUNT(*) OVER (
28         PARTITION BY r.REPORT_LOCATION
29         ORDER BY r.report_date
30         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
31     ) as cumulative_count
32 FROM reports r
33 JOIN crops c ON r.crop_id = c.crop_id;

View created.

SQL> /

View created.

SQL> _
```

```
SQL Plus

24     ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
25 ) as moving_avg_severity,
26 -- Cumulative count
27 COUNT(*) OVER (
28     PARTITION BY r.REPORT_LOCATION
29     ORDER BY r.report_date
30     ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
31 ) as cumulative_count
32 FROM reports r
33 JOIN crops c ON r.crop_id = c.crop_id;

View created.

SQL> /

View created.

SQL> CREATE OR REPLACE PROCEDURE show_window_function_examples IS
2  CURSOR c_rankings IS
3      SELECT report_id, REPORT_LOCATION, severity, severity_rank,
4             prev_severity, next_severity, moving_avg_severity
5      FROM v_report_rankings
6      WHERE ROWNUM <= 10;
7  BEGIN
8      DBMS_OUTPUT.PUT_LINE('=== Window Function Examples ===');
9      DBMS_OUTPUT.PUT_LINE('');
10
11     FOR rec IN c_rankings LOOP
12         DBMS_OUTPUT.PUT_LINE('Report: ' || rec.report_id);
13         DBMS_OUTPUT.PUT_LINE(' Location: ' || rec.REPORT_LOCATION);
14         DBMS_OUTPUT.PUT_LINE(' Current Severity: ' || rec.severity);
15         DBMS_OUTPUT.PUT_LINE(' Rank: ' || rec.severity_rank);
16         DBMS_OUTPUT.PUT_LINE(' Previous: ' || rec.prev_severity);
17         DBMS_OUTPUT.PUT_LINE(' Next: ' || rec.next_severity);
18         DBMS_OUTPUT.PUT_LINE(' Moving Avg: ' || ROUND(rec.moving_avg_severity, 2));
19         DBMS_OUTPUT.PUT_LINE('---');
20     END LOOP;
21 END show_window_function_examples;
22 /

Procedure created.

SQL>
```

SQL Plus

```
Rank: 1
Previous: 8
Next: 2
Moving Avg: 9.33
---
Report: 340
Location: Huye
Current Severity: 10
Rank: 1
Previous: 2
Next: 4
Moving Avg: 4.67
---
Report: 265
Location: Huye
Current Severity: 10
Rank: 1
Previous: 4
Next: 10
Moving Avg: 8
---
Report: 295
Location: Huye
Current Severity: 10
Rank: 1
Previous: 10
Next: 8
Moving Avg: 8
---
TEST 11: Process Reports with Bulk Collect
Fetched 100 pending reports
Processing report 378 with severity 7
Processing report 379 with severity 2
Processing report 380 with severity 10
Processing report 382 with severity 5
Processing report 383 with severity 4
=====
ALL TESTS COMPLETED SUCCESSFULLY
=====
PL/SQL procedure successfully completed.
SQL> _
```

```

SQL Plus
4      holiday_name VARCHAR2(100) NOT NULL,
5      description VARCHAR2(500),
6      created_at DATE DEFAULT SYSDATE
7  );

Table created.

SQL>
SQL> CREATE SEQUENCE seq_holidays START WITH 1 NOCACHE;

Sequence created.

SQL>
SQL> -- Insert Rwanda Public Holidays for upcoming months (December 2025 - January 2026)
SQL> INSERT INTO public_holidays(holiday_id, holiday_date, holiday_name, description)
2  VALUES (seq_holidays.NEXTVAL, DATE '2025-12-25', 'Christmas Day', 'Christian holiday');

1 row created.

SQL>
SQL> INSERT INTO public_holidays(holiday_id, holiday_date, holiday_name, description)
2  VALUES (seq_holidays.NEXTVAL, DATE '2025-12-26', 'Boxing Day', 'Day after Christmas');

1 row created.

SQL>
SQL> INSERT INTO public_holidays(holiday_id, holiday_date, holiday_name, description)
2  VALUES (seq_holidays.NEXTVAL, DATE '2026-01-01', 'New Year Day', 'First day of the year');

1 row created.

SQL>
SQL> INSERT INTO public_holidays(holiday_id, holiday_date, holiday_name, description)
2  VALUES (seq_holidays.NEXTVAL, DATE '2026-01-04', 'Patriotic Day', 'Commemoration of fallen heroes');

1 row created.

SQL>
SQL> COMMIT;

Commit complete.

SQL>

```

```

SQL> CREATE TABLE holidays (
2      holiday_id      NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
3      holiday_date    DATE NOT NULL,
4      description     VARCHAR2(200)
5  );

Table created.

SQL> INSERT INTO holidays (holiday_date, description)
2  VALUES (DATE '2025-01-01', 'New Year Holiday');

1 row created.

SQL>
SQL> INSERT INTO holidays (holiday_date, description)
2  VALUES (DATE '2025-01-05', 'Heroes Day');

1 row created.

SQL>
SQL> COMMIT;

Commit complete.

SQL> CREATE OR REPLACE FUNCTION log_action(
2      p_user      VARCHAR2,
3      p_action    VARCHAR2,
4      p_status    VARCHAR2,
5      p_reason    VARCHAR2
6  ) RETURN NUMBER IS
7      v_id NUMBER;
8  BEGIN
9      v_id := seq_audit.NEXTVAL;
10
11      INSERT INTO audit_log(
12          log_id, user_id, action, object_type, object_id, status, notes, created_at
13      )
14      VALUES (

```

```

12/3 PLS 00221: LOG_ACTION is not a procedure or is undefined
SQL> CREATE OR REPLACE TRIGGER trg_reports_compound
2 FOR UPDATE OR DELETE ON reports
3 COMPOUND TRIGGER
4
5     v_status VARCHAR2(200);
6     v_dummy NUMBER; -- <-- Capture return from log_action
7
8 BEFORE EACH ROW IS
9 BEGIN
10     v_status := check_restriction();
11
12     IF v_status LIKE 'DENIED%' THEN
13         v_dummy := log_action(USER, 'MODIFY_REPORT', 'DENIED', v_status);
14         RAISE_APPLICATION_ERROR(-20051, v_status);
15     ELSE
16         v_dummy := log_action(USER, 'MODIFY_REPORT', 'ALLOWED', 'Update/Delete allowed.');

Trigger created.



SQL> _


```

SQL Plus

```

20 WHERE TRUNC(holiday_date) = TRUNC(SYSDATE);
21 EXCEPTION
22 WHEN OTHERS THEN
23     v_is_holiday := 'NO';
24 END;
25
26 -- Match the actual table structure
27 INSERT INTO trigger_audit_log(
28     audit_id,
29     table_name,
30     operation,
31     username,
32     operation_date,
33     day_of_week,
34     is_holiday,
35     is_allowed,
36     denial_reason
37 )
38 VALUES (
39     v_id,
40     'REPORTS',
41     p_action,
42     p_user,
43     SYSDATE,
44     TRIM(v_day),
45     v_is_holiday,
46     p_status,
47     p_reason
48 );
49
50 COMMIT;
51 RETURN v_id;
52
53 EXCEPTION
54 WHEN OTHERS THEN
55     ROLLBACK;
56     RETURN -1;
57 END;
58 /

```

Function created.

SQL>

```
62      EXCEPTION
63      WHEN OTHERS THEN
64          DBMS_OUTPUT.PUT_LINE('ⓓ DELETE BLOCKED');
65          DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
66          ROLLBACK;
67      END;
68
69      DBMS_OUTPUT.PUT_LINE('');
70      DBMS_OUTPUT.PUT_LINE('=====');
71      DBMS_OUTPUT.PUT_LINE('TESTING COMPLETED');
72      DBMS_OUTPUT.PUT_LINE('=====');
73      DBMS_OUTPUT.PUT_LINE('');
74      DBMS_OUTPUT.PUT_LINE('View audit log with:');
75      DBMS_OUTPUT.PUT_LINE('SELECT * FROM trigger_audit_log ORDER BY audit_id DESC;');
76  END;
77  /
=====
TRIGGER TESTING - PHASE VII
=====
Current Date: 2025-12-06
Day of Week: SAT
>>> TODAY IS A WEEKEND - OPERATIONS WILL BE ALLOWED <<<
=====
TEST 1: Attempt INSERT on REPORTS
=====
? INSERT ALLOWED - Record inserted successfully
=====
TEST 2: Attempt UPDATE on REPORTS
=====
? UPDATE ALLOWED - Record updated successfully
=====
TEST 3: Attempt DELETE on REPORTS
=====
? DELETE ALLOWED - Record deleted successfully
=====
TESTING COMPLETED
=====
View audit log with:
SELECT * FROM trigger_audit_log ORDER BY audit_id DESC;

PL/SQL procedure successfully completed.

SQL> _
```

```

4      TO_CHAR(SYSDATE, 'DY', 'NLS_DATE_LANGUAGE=ENGLISH') as day_abbr,
5      CASE
6          WHEN TO_CHAR(SYSDATE, 'DY', 'NLS_DATE_LANGUAGE=ENGLISH') IN ('MON','TUE','WED','THU','FRI')
7              THEN 'WEEKDAY (BLOCKED)'
8              ELSE 'WEEKEND (ALLOWED)'
9      END as day_status,
10     CASE
11         WHEN EXISTS (SELECT 1 FROM holidays WHERE TRUNC(holiday_date) = TRUNC(SYSDATE))
12             THEN 'YES (BLOCKED)'
13             ELSE 'NO'
14     END as is_holiday
15 FROM dual;

CURRENT_DA DAY_OF_WEEK          DAY_ABBR    DAY_STATUS
-----
IS_HOLIDAY
-----
2025-12-06 SATURDAY            SAT        WEEKEND (ALLOWED)
NO

SQL>
SQL> -- Query 6: View all holidays
SQL> SELECT
2     holiday_id,
3     TO_CHAR(holiday_date, 'YYYY-MM-DD') as holiday_date,
4     TO_CHAR(holiday_date, 'DAY') as day_of_week,
5     description
6 FROM holidays
7 ORDER BY holiday_date;

HOLIDAY_ID HOLIDAY_DA DAY_OF_WEEK
-----
DESCRIPTION
-----
1 2025-01-01 WEDNESDAY
New Year Holiday
2 2025-01-05 SUNDAY
Heroes Day

SQL>

```

```

Function created.

SQL> CREATE OR REPLACE FUNCTION get_disease_name(p_disease_id IN NUMBER) RETURN VARCHAR2 IS
2     v_name VARCHAR2(100);
3 BEGIN
4     SELECT disease_name INTO v_name
5     FROM diseases
6     WHERE disease_id = p_disease_id;
7
8     RETURN v_name;
9
10 EXCEPTION
11     WHEN NO_DATA_FOUND THEN
12         RETURN 'Unknown Disease';
13     WHEN OTHERS THEN
14         RETURN 'Error';
15 END get_disease_name;
16 /

Function created.

SQL> CREATE OR REPLACE FUNCTION get_avg_severity_by_location(
2     p_location IN VARCHAR2
3 ) RETURN NUMBER IS
4     v_avg NUMBER;
5 BEGIN
6     SELECT NVL(AVG(severity), 0)
7     INTO v_avg
8     FROM reports
9     WHERE REPORT_LOCATION = p_location;
10
11     RETURN ROUND(v_avg, 2);
12
13 EXCEPTION
14     WHEN OTHERS THEN
15         RETURN 0;
16 END get_avg_severity_by_location;
17 /

Function created.

SQL>

```

```
SQL Plus
-----
AUDIT_ID OPERATION          TIME
-----
DENIAL_REASON
-----
6 TEST_ACTION              2025-12-06 16:38:35
Testing

SQL>
SQL> -- Current day status
SQL> SELECT
  2   TO_CHAR(SYSDATE, 'YYYY-MM-DD') as date,
  3   TO_CHAR(SYSDATE, 'DAY') as day_of_week,
  4   CASE
  5     WHEN TO_CHAR(SYSDATE, 'DY', 'NLS_DATE_LANGUAGE=ENGLISH') IN ('MON','TUE','WED','THU','FRI')
  6       THEN 'WEEKDAY (BLOCKED)'
  7     ELSE 'WEEKEND (ALLOWED)'
  8   END as status
  9 FROM dual;
      TO_CHAR(SYSDATE, 'YYYY-MM-DD') as date,
      *
ERROR at line 2:
ORA-00923: FROM keyword not found where expected
Help: https://docs.oracle.com/error-help/db/ora-00923/

SQL> -- Current day status
SQL> SELECT
  2   TO_CHAR(SYSDATE, 'YYYY-MM-DD') as current_date,
  3   TO_CHAR(SYSDATE, 'DAY') as day_of_week,
  4   CASE
  5     WHEN TO_CHAR(SYSDATE, 'DY', 'NLS_DATE_LANGUAGE=ENGLISH') IN ('MON','TUE','WED','THU','FRI')
  6       THEN 'WEEKDAY (BLOCKED)'
  7     ELSE 'WEEKEND (ALLOWED)'
  8   END as status
  9 FROM dual;

CURRENT_DA DAY_OF_WEEK          STATUS
-----
2025-12-06 SATURDAY              WEEKEND (ALLOWED)

SQL>
```